App Dev 2 Project Report

1. Student Details:

Name: S. Krishna Subramanian
Roll Number: 23f1000655
Email: 23f1000655@ds.study.iitm.ac.in

I am a student of the IIT Madras BS Degree program with a keen interest in full-stack web application development. I enjoy designing systems that solve real-world problems by combining backend engineering, data management, and intuitive user interfaces.

2. Project Details:

Project Title: HireSphere - Campus Recruitment Portal

Problem Statement: Placement Portal Application - V2 - To design and build a multi-role web application that streamlines the campus placement process for students, companies, and placement administrators - replacing manual, fragmented coordination with a unified digital platform.

Approach: HireSphere was built using Flask as the backend framework with role-based access control (admin, student, company) enforced via Flask-Security. The frontend is a single-page application powered by Vue 3 (CDN, no build step) and Vue Router 4. The application supports three distinct dashboards with separate capabilities:

- **Admin:** approves/rejects companies, blacklists users, monitors all drives and applications, views a live dashboard with placement statistics.
- **Company:** registers, posts placement drives after admin approval, manages applications, updates candidate statuses (shortlisted / selected / rejected).
- **Student:** browses approved companies and drives, applies with PDF resume upload, tracks application history with real-time status updates.

Background jobs are handled via Celery with Redis as the message broker - including daily deadline reminder emails to students, automated monthly placement reports to the admin, and an asynchronous CSV export of a student's application history delivered via email. Redis-based caching (Flask-Caching) is implemented across all major read endpoints to reduce database load and improve API response times.

3. AI/LLM Declaration:

I used Claude AI as an AI/LLM assistant during the development of this project. The overall extent of AI/LLM usage is approximately 30-35%, limited to the following specific areas:

- Formation of basic code architecture - initial scaffolding of the Flask application factory pattern, blueprint structure, and SQLAlchemy model definitions.
- Error handling suggestions — identifying edge cases in API routes such as missing profile checks, duplicate application prevention, and foreign key deletion ordering.

- Celery scheduling errors — debugging Celery Beat configuration issues, task registration errors, and Redis broker connectivity problems in a WSL environment.
- CSS styling guidance — assistance with layout decisions, Bootstrap 5 utility class usage, and colour scheme consistency across the Vue frontend.

All core business logic, database design, eligibility filtering, frontend Vue component structure, email template design, and final integration and testing were done manually by me.

4. Frameworks and Libraries Used:

Technology / Library	Purpose
Flask	Core backend web framework
Flask-Security-Too	Role-based authentication and session management
SQLAlchemy	ORM for SQLite database interaction
Flask-Mail	SMTP email sending for notifications and reports
Flask-Caching	Redis-backed API response caching with TTL expiry
Celery + Redis	Async task queue for background jobs and scheduling
Vue 3 + Vue Router 4	Frontend SPA framework (CDN, no build step)
Bootstrap 5	Frontend styling and responsive UI components
Axios	HTTP client for Vue-to-Flask API communication
Werkzeug	Password hashing and file upload handling
SQLite	Lightweight relational database

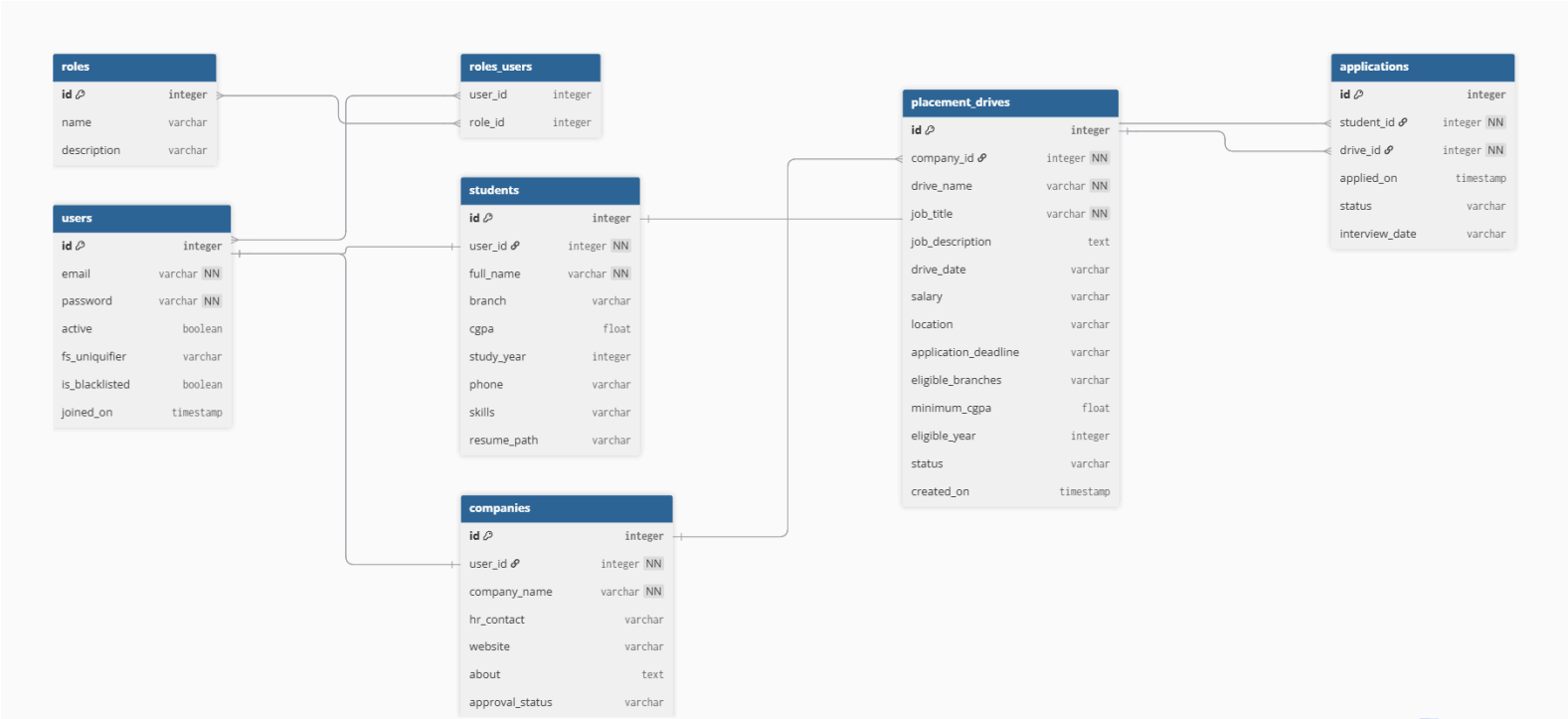
5. Database Schema / ER Diagram:

The database consists of five core tables with the following relationships:

User	Student	Company	PlacementDrive	Application
id (PK)	id (PK)	id (PK)	id (PK)	id (PK)
email	user_id (FK)	user_id (FK)	company_id (FK)	student_id (FK)
password	full_name	company_name	drive_name	drive_id (FK)
active	branch	approval_status	job_title	applied_on
is_blacklisted	cgpa	hr_contact	application_deadline	status
roles (M2M)	resume_path	website	status	interview_date

Relationships:

- Many-to-Many → User ↔ Role (via roles_users junction table)
- One-to-One → User → Student
- One-to-One → User → Company
- One-to-Many → Company → PlacementDrive
- One-to-Many → Student → Application
- One-to-Many → PlacementDrive → Application



6. API Resource Endpoints:

Endpoint	Method	Description
`/api/auth/register`	POST	Register a new student or company account
`/api/auth/login`	POST	Authenticate user and return session token
`/api/auth/logout`	POST	Invalidate current session
`/api/admin/stats`	GET	Dashboard statistics (students, companies, drives)
`/api/admin/pending-companies`	GET	List companies awaiting approval
`/api/admin/approve-company/<id>`	POST	Approve a company registration
`/api/admin/reject-company/<id>`	POST	Reject a company registration
`/api/admin/blacklist-company/<id>`	POST	Blacklist a company and cancel their drives
`/api/admin/blacklist-student/<id>`	POST	Blacklist a student account
`/api/admin/mark-drive-complete/<id>`	POST	Mark a placement drive as completed
`/api/admin/students`	GET	List all registered students
`/api/admin/ongoing-drives`	GET	List all active placement drives
`/api/admin/applications`	GET	List all submitted applications
`/api/company/my-drives`	GET	List drives posted by current company
`/api/company/create-drive`	POST	Post a new placement drive
`/api/company/drive/<id>/applications`	GET	View all applicants for a specific drive
`/api/company/application/<id>/update-status`	POST	Update a single application status
`/api/company/bulk-update-statuses`	POST	Bulk update multiple application statuses

`/api/student/companies`	GET	Browse all approved companies
`/api/student/company/<id>`	GET	View company details and open drives
`/api/student/apply/<drive_id>`	POST	Apply to a drive with resume upload
`/api/student/my-applications`	GET	View student's own application list
`/api/student/history`	GET	Full placement history with status
`/api/student/export-csv`	POST	Trigger async CSV export via email (Celery)

7. Video Presentation:

https://drive.google.com/file/d/18qNSd7D6VTGOtK6pzlKL1mLGLRsLqoXn/view?usp=drive_link